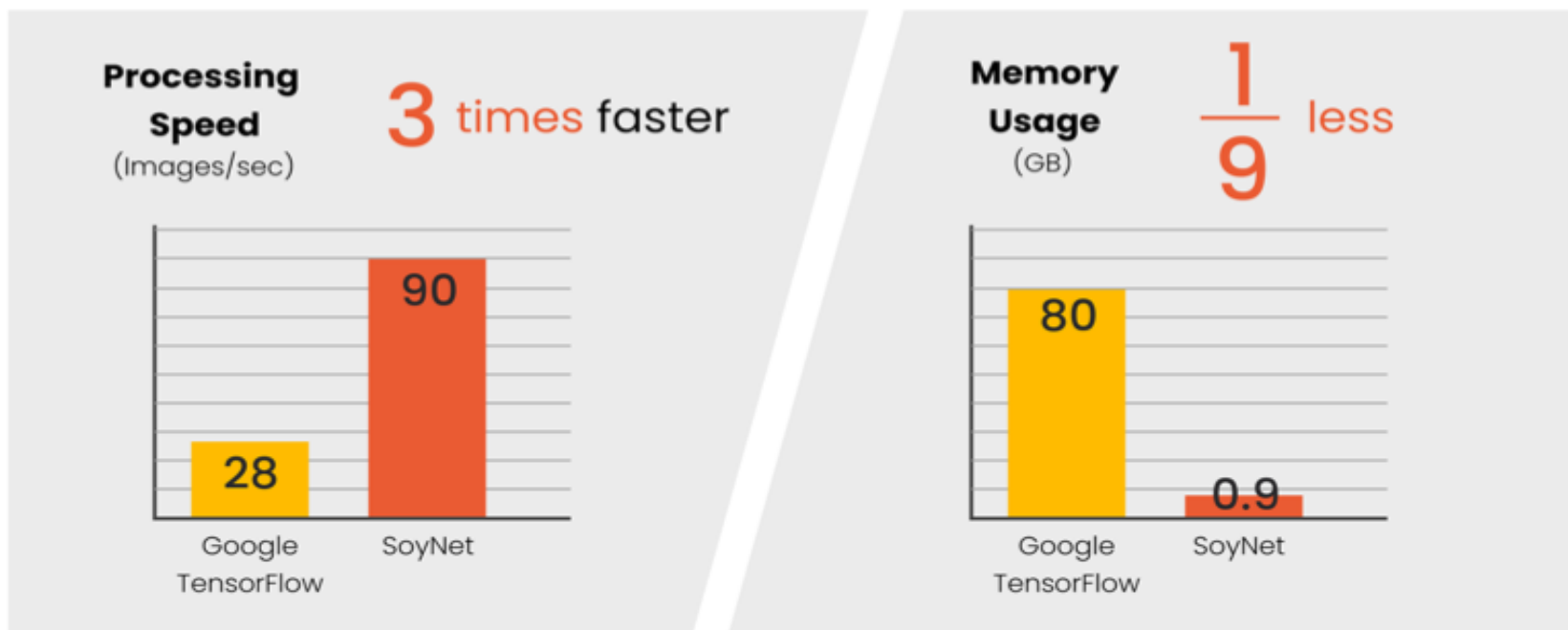


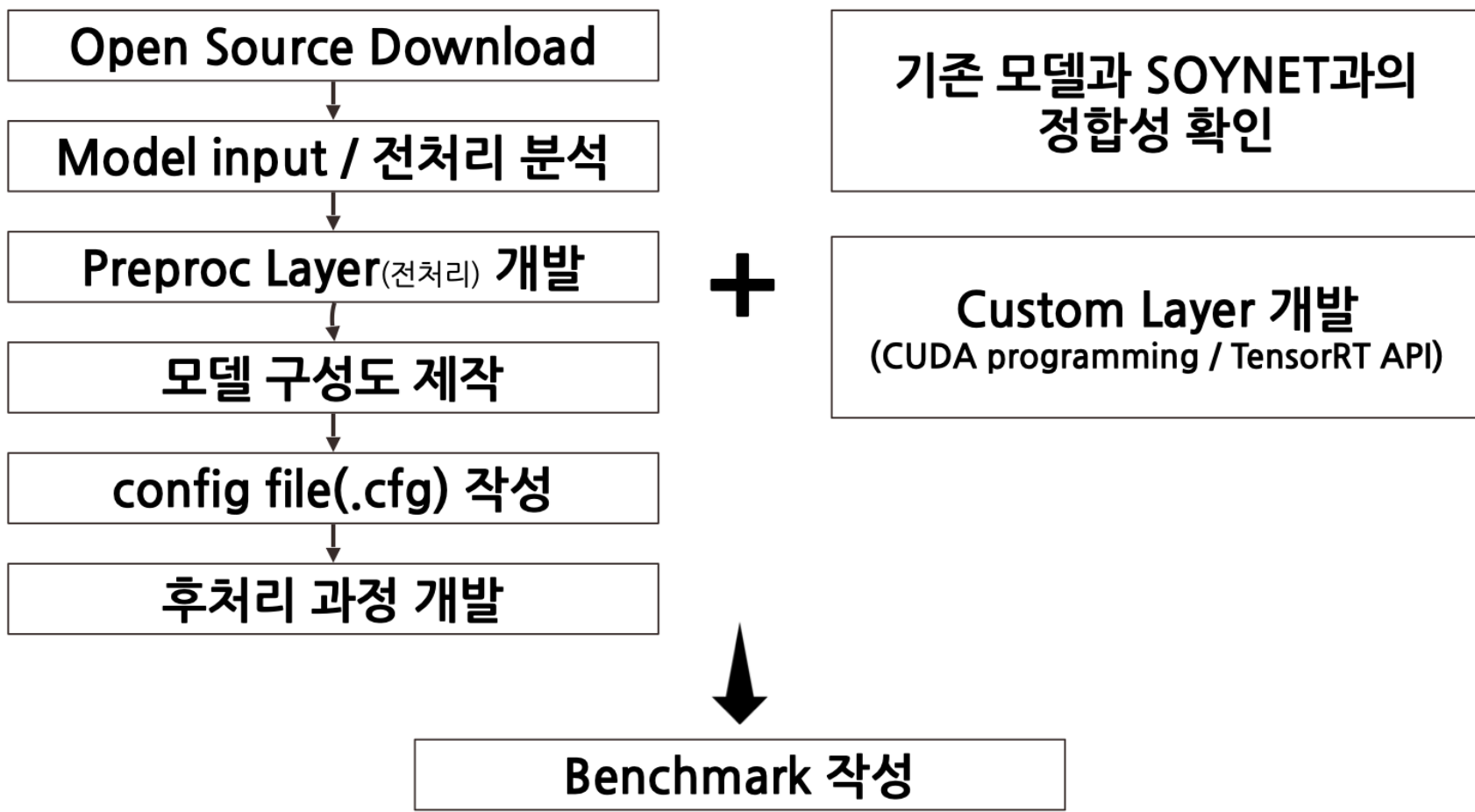
# DL inference 엔진 최적화

# 목표



AI 모델의 대규모 서비스에서의 실시간 성능을 위한,  
딥러닝 추론 엔진 최적화가 목표였다!

# 개발 프로세스



# Open Source Download

 [Get Started](#) [Ecosystem](#) [Mobile](#) [Blog](#) [Tutorials](#) [Docs](#) [Resources](#) [GitHub](#)



## VGG-NETS

By Pytorch Team

Award winning ConvNets from 2014 Imagenet ILSVRC challenge

[View on Github](#) [Open on Google Colab](#) [Open Model Demo](#)

| ConvNet Configuration       |                  |                  |                  |                  |                  |
|-----------------------------|------------------|------------------|------------------|------------------|------------------|
| A                           | A-LRN            | B                | C                | D                | E                |
| 11 weight layers            | 11 weight layers | 13 weight layers | 16 weight layers | 16 weight layers | 19 weight layers |
| Input (224 x 224 RGB image) |                  |                  |                  |                  |                  |
| conv3-64                    | conv3-64<br>LRN  | conv3-64         | conv3-64         | conv3-64         | conv3-64         |
| maxpool                     |                  |                  |                  |                  |                  |
| conv3-128                   | conv3-128        | conv3-128        | conv3-128        | conv3-128        | conv3-128        |
| maxpool                     |                  |                  |                  |                  |                  |
| conv3-256                   | conv3-256        | conv3-256        | conv3-256        | conv3-256        | conv3-256        |
| conv3-256                   | conv3-256        | conv3-256        | conv3-256        | conv3-256        | conv3-256        |
| maxpool                     |                  |                  |                  |                  |                  |
| conv3-512                   | conv3-512        | conv3-512        | conv3-512        | conv3-512        | conv3-512        |
| conv3-512                   | conv3-512        | conv3-512        | conv3-512        | conv3-512        | conv3-512        |
| maxpool                     |                  |                  |                  |                  |                  |
| conv3-512                   | conv3-512        | conv3-512        | conv3-512        | conv3-512        | conv3-512        |
| conv3-512                   | conv3-512        | conv3-512        | conv3-512        | conv3-512        | conv3-512        |
| maxpool                     |                  |                  |                  |                  |                  |
| FC-4096                     |                  |                  |                  |                  |                  |
| FC-4096                     |                  |                  |                  |                  |                  |
| FC-1000                     |                  |                  |                  |                  |                  |
| softmax                     |                  |                  |                  |                  |                  |

```
import torch
model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg11', pretrained=True)
# or any of these variants
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg11_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg13', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg13_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg16', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg16_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg19', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg19_bn', pretrained=True)
model.eval()
```

All pre-trained models expect input images normalized in the same way, i.e. mini-batches of 3-channel RGB images of shape  $(3 \times H \times W)$ , where  $H$  and  $W$  are expected to be at least 224. The images have to be

먼저, 오픈소스 모델을 분석했다.

나는 다음 모델들을 담당했다.

**분류 :**

- VGG16/19, EfficientNet-B6

**Segmentation :**

- FCN

**객체 탐지 :**

- YOLOX-X

**Pose 예측 :**

- Posenet Transpose-h-a4

**언어 :**

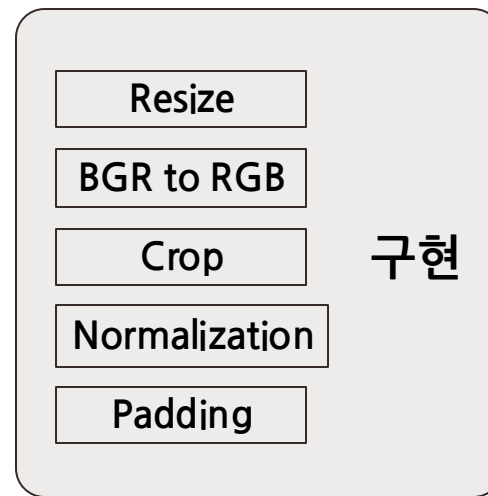
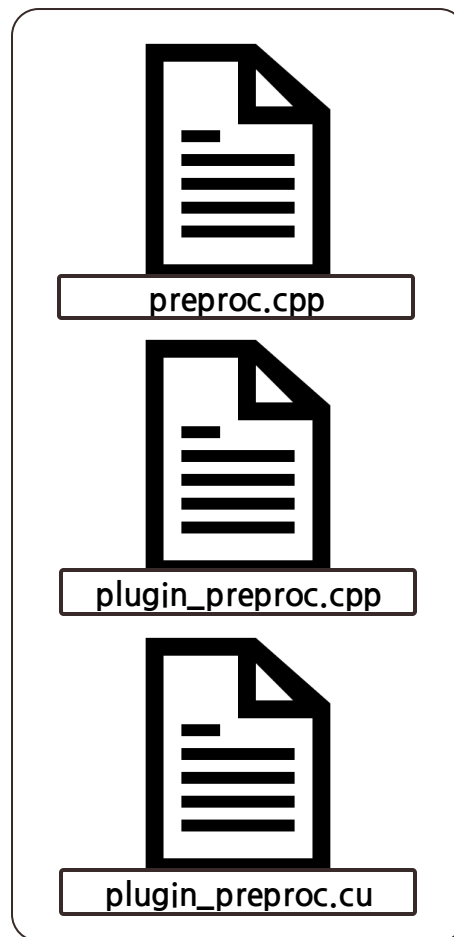
- Bart\_Paragraph

# 전처리 분석/개발

```
from torchvision import transforms
input_image = Image.open(filename) input_image: <PIL.JpegImagePlugin.JpegImageFile im
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
input_tensor = preprocess(input_image)
input_batch = input_tensor.unsqueeze(0) # create a mini-batch as expected by the model

# move the input and model to GPU for speed if available
if torch.cuda.is_available():
    input_batch = input_batch.to('cuda')
    model.to('cuda')

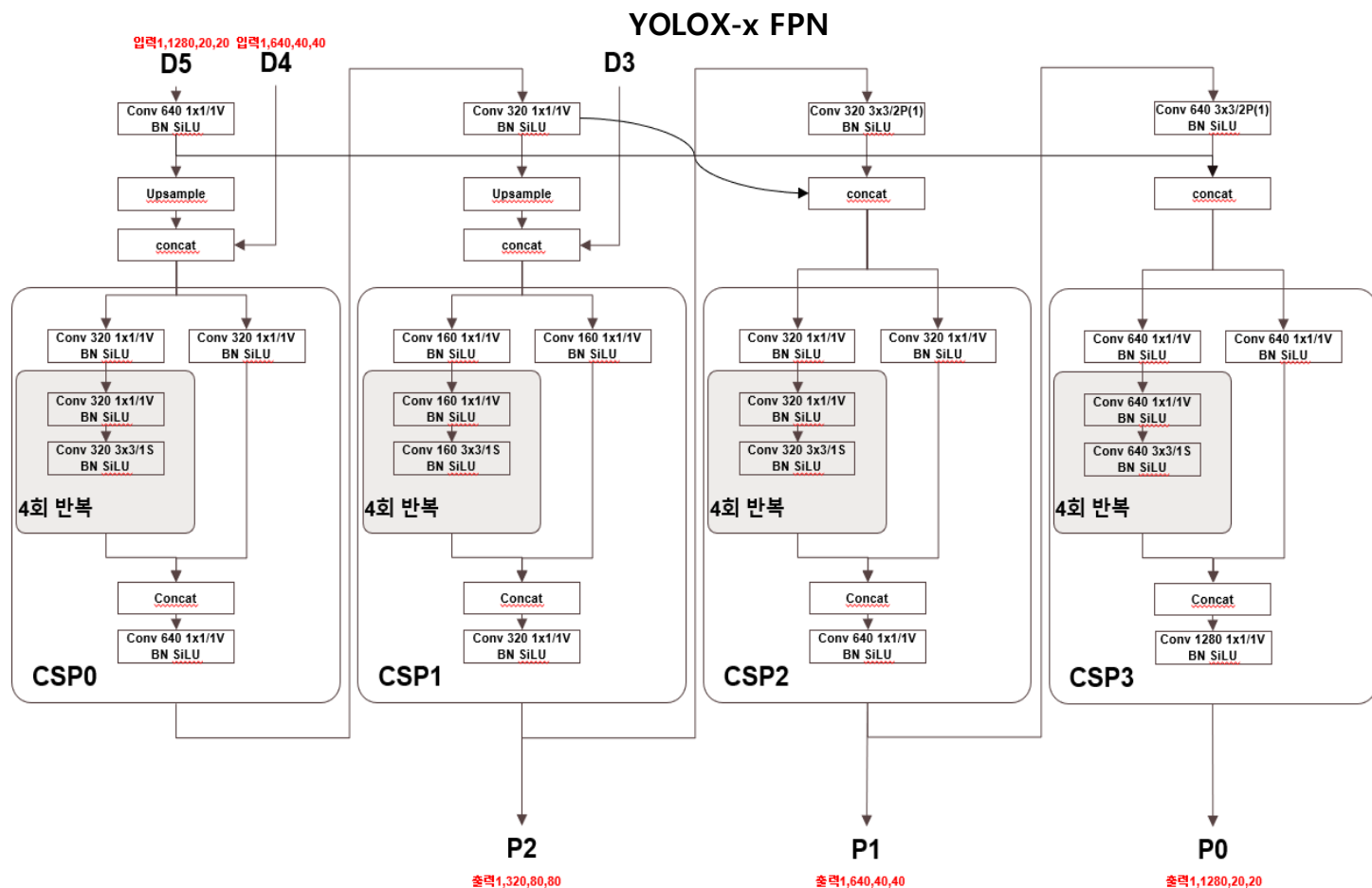
with torch.no_grad():
```



오픈소스의 입력 파이프라인을 디버깅해 전처리 과정을 분석했다.

이를 GPU 최적화 형태로 모듈화해 **C++/CUDA** 기반으로 구현했다.

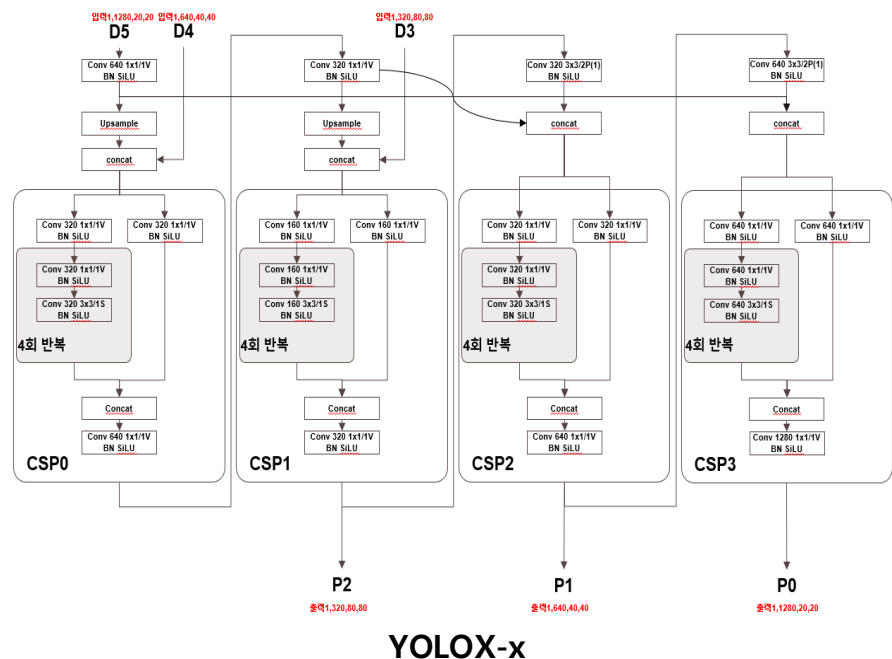
# 모델 구성도 제작



모델 실행을 디버깅해,  
연산 단위를 세밀히 추적했다.

이를 기반으로 구성도(DAG 형태)를 작성,  
모델의 연산 경로와 레이어 특성을 시각화했다.

# Config file(.cfg) 작성



```
bot-sort-yolox-x.cfg  X bot-sort-baseline.cfg  bot-sort.cpp  nms.cpp
172 [conv2d] input=P2 channel=320 refname=STEM0
173 [conv2d] input=P1 channel=320 refname=STEM1
174 [conv2d] input=P0 channel=320 refname=STEM2
175 ## CLS conv ##
176 [conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=CLS_0
177 [conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=CLS_1
178 [conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=CLS_2
179 ## REG conv ##
180 [conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=REG_0
181 [conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=REG_1
182 [conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_tf=same refname=REG_2
183
184 ## CLS PRED ##
185 [conv2d] input=CLS_0 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_0
186 [conv2d] input=CLS_1 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_1
187 [conv2d] input=CLS_2 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_2
188 ## REG PRED ##
189 [conv2d] input=REG_0 channel=4 weight_order=wa act=identity refname=REGP_0
190 [conv2d] input=REG_1 channel=4 weight_order=wa act=identity refname=REGP_1
191 [conv2d] input=REG_2 channel=4 weight_order=wa act=identity refname=REGP_2
192 ## OBJ PRED ##
193 [conv2d] input=REGP_0 channel=1 weight_order=wa act=sigmoid refname=OBJP_0
194 [conv2d] input=REGP_1 channel=1 weight_order=wa act=sigmoid refname=OBJP_1
195 [conv2d] input=REGP_2 channel=1 weight_order=wa act=sigmoid refname=OBJP_2
196
197 [concat] input=REGP_0,OBJP_0,CLSP_0 axis=1 [reshape] shape=$BATCH_SIZE,$CLASS_COUNT+5,-1 [trans] order=0,2,1 ref
198 [concat] input=REGP_1,OBJP_1,CLSP_1 axis=1 [reshape] shape=$BATCH_SIZE,$CLASS_COUNT+5,-1 [trans] order=0,2,1 ref
199 [concat] input=REGP_2,OBJP_2,CLSP_2 axis=1 [reshape] shape=$BATCH_SIZE,$CLASS_COUNT+5,-1 [trans] order=0,2,1 ref
200 #[concat] input=OUT_0,OUT_1,OUT_2 axis=1 refname=OUTPUT #shape:1,23625,6
201
202 ### decode ###
203 [yolo] input=OUT_0 mode=yolox class_count=$CLASS_COUNT height=100 width=180 stride=8 conf_thres=$CONF_THRES ref
204 [yolo] input=OUT_1 mode=yolox class_count=$CLASS_COUNT height=50 width=90 stride=16 conf_thres=$CONF_THRES refna
205 [yolo] input=OUT_2 mode=yolox class_count=$CLASS_COUNT height=25 width=45 stride=32 conf_thres=$CONF_THRES refna
206 [concat] input=B0,B1,B2 axis=1 refname=B
207 [concat] input=C0,C1,C2 axis=1 refname=C
208
209 [nms] input=B,C mode=yolox class_count=$CLASS_COUNT region_count=$REGION_COUNT nms_count=$NMS_COUNT conf_thres=$
210 [postproc] mode=bot-sort arg_v132=$INPUT_SIZE,$IMG_SIZE
211 refname=FIN [output] refname=FIN [end]
```

구성도를 바탕으로,  
엔진 문법에 맞춘 Layer 단위 Config 파일을 작성했다.

각 Layer를 작성할 때마다 기존 모델과 정합성을 즉시 검증,  
오차 누적 없이 동일한 연산 결과를 확보했다.

yolox.cfg

# 후처리 분석/개발

```
__global__ void kernel_yolox_postproc(float* output, float* input, float gain, int tcount)
{
    int pos = threadIdx.x + blockIdx.x * blockDim.x;
    if (pos >= tcount) return;

    int base = pos * 6;
    float x1 = input[base + 0];
    float y1 = input[base + 1];
    float x2 = input[base + 2];
    float y2 = input[base + 3];

    x1 /= gain;
    y1 /= gain;
    x2 /= gain;
    y2 /= gain;

    output[base + 0] = (float)((int)x1);
    output[base + 1] = (float)((int)y1);
    output[base + 2] = (float)((int)x2);
    output[base + 3] = (float)((int)y2);

    output[base + 4] = input[base + 4];
    output[base + 5] = input[base + 5];
}

void cuda_yolox_postproc(float* output, float* input, float gain, int N, int bbox_count, cudaStream_t cudaStream)
{
    int tcount = N * bbox_count;
    int block = 64;
    int grid = (tcount - 1) / block + 1;

    kernel_yolox_postproc << grid, block, 0, cudaStream >> (output, input, gain, tcount);
}
```

yolox\_postproc.cu

오픈소스 모델을 디버깅하여 출력 구조를 정밀 분석했다.

모델 특성에 맞는 후처리 로직(탐지/Segmentation/분류)을  
cpp 기반 모듈과 postproc layer로 직접 개발했다.

# 성과



## YOLOX-X

### FPS 향상

기존 모델의 경우 평균 **16~17 FPS**였으나, 최적화 후 **38~39 FPS**로 약 **2.3배 속도 개선** 달성했다.

### 메모리 최적화

GPU 메모리 사용량을 **1800MB** → **800MB**로 절감하여, 약 **55% 메모리 사용량 감소**했다.